

SIMATS ENGINEERING ASSESSMENT TOOL – 2

DATABASE MANAGEMENT SYSTEMS (CSA05) CO5 – Pseudocode Writing Task (Algorithm Design)

Total Marks: 50 | Each Question: 5 Marks

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

The algorithm accepts an SQL query, checks its syntax and semantics, creates a logical query plan, optimizes it, executes the selected physical plan, and returns the result.

Pseudocode:

```
declare SQL_Query = "SELECT * FROM Pets"

print "Step 1: SQL Query Received"

print SQL_Query

print "Step 2: Parsing the Query"

print "Step 3: Checking Syntax and Semantics"

print "Step 4: Creating Logical Query Plan"

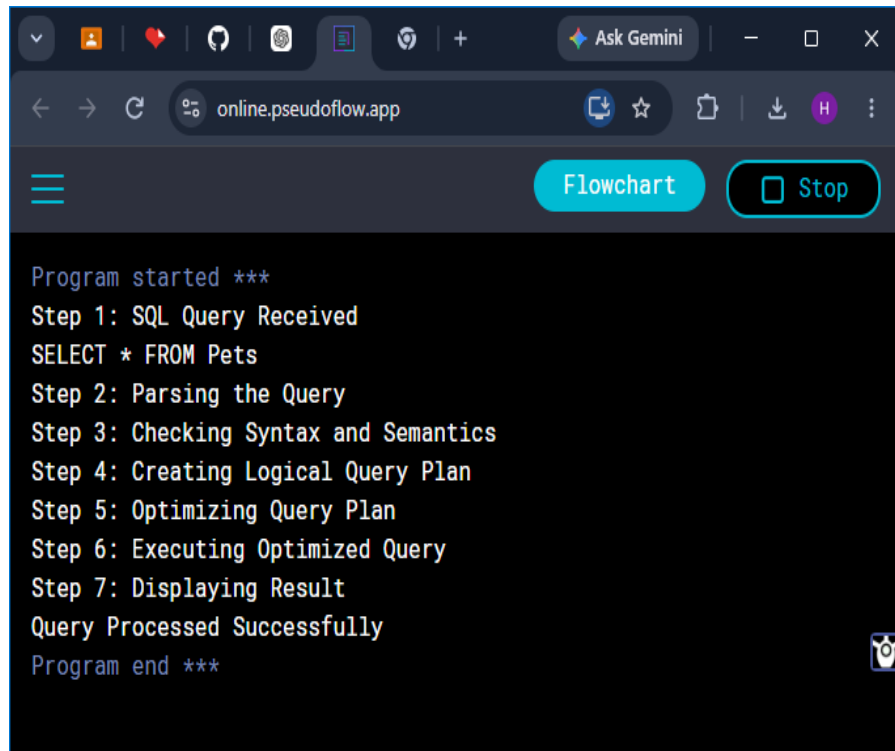
print "Step 5: Optimizing Query Plan"

print "Step 6: Executing Optimized Query"

print "Step 7: Displaying Result"

print "Query Processed Successfully"
```

OUTPUT



```
Program started ***
Step 1: SQL Query Received
SELECT * FROM Pets
Step 2: Parsing the Query
Step 3: Checking Syntax and Semantics
Step 4: Creating Logical Query Plan
Step 5: Optimizing Query Plan
Step 6: Executing Optimized Query
Step 7: Displaying Result
Query Processed Successfully
Program end ***
```

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

The optimizer evaluates alternative join orders, estimates their I/O costs using relation statistics, and selects the plan with the minimum estimated cost.

Pseudocode:

```
declare pages_R = 100
declare pages_S = 50

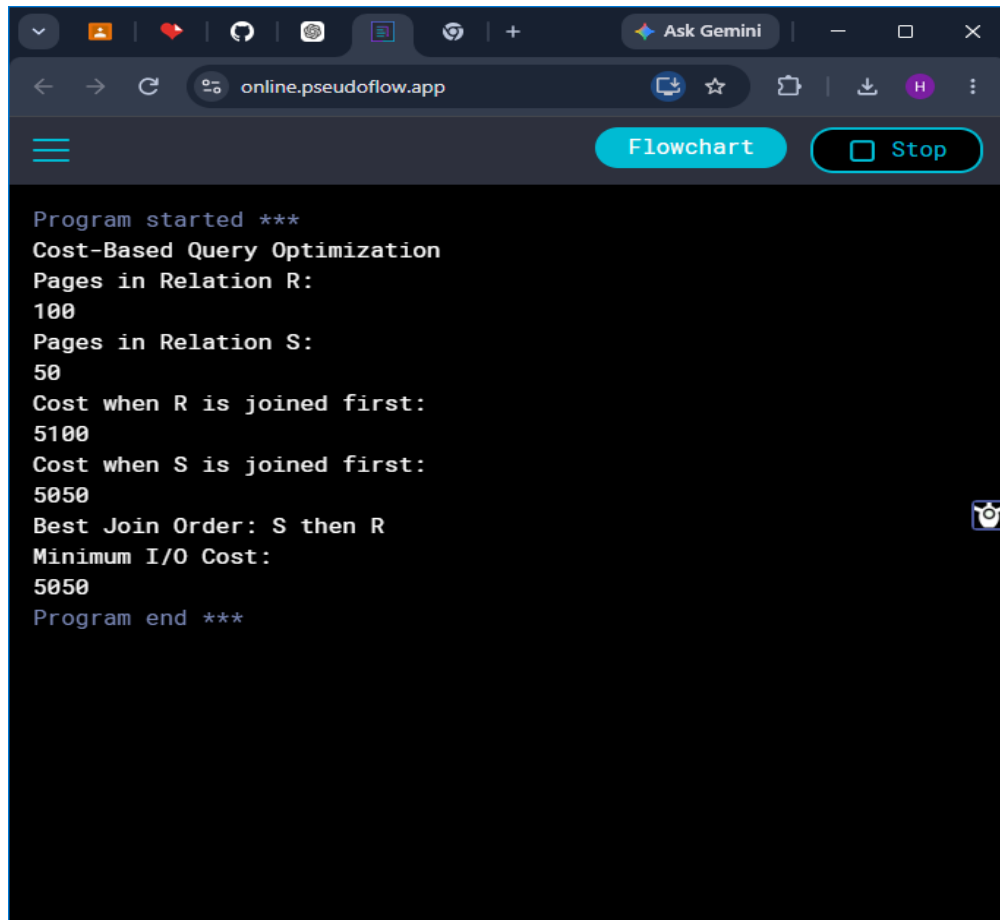
declare cost_R_first = pages_R + (pages_R * pages_S)
declare cost_S_first = pages_S + (pages_S * pages_R)

print "Cost-Based Query Optimization"

print "Pages in Relation R:"
print pages_R
```

```
print "Pages in Relation S:"  
print pages_S  
  
print "Cost when R is joined first:"  
print cost_R_first  
  
print "Cost when S is joined first:"  
print cost_S_first  
  
if (cost_R_first < cost_S_first)  
    print "Best Join Order: R then S"  
    print "Minimum I/O Cost:"  
    print cost_R_first  
else  
    print "Best Join Order: S then R"  
    print "Minimum I/O Cost:"  
    print cost_S_first  
endif
```

OUTPUT



The screenshot shows a web browser window with the address bar displaying "online.pseudoflow.app". The browser's top bar includes a search bar with "Ask Gemini" and several icons. The page has a dark theme. At the top, there is a hamburger menu icon on the left, a "Flowchart" button in the center, and a "Stop" button on the right. The main content area displays the following text:

```
Program started ***
Cost-Based Query Optimization
Pages in Relation R:
100
Pages in Relation S:
50
Cost when R is joined first:
5100
Cost when S is joined first:
5050
Best Join Order: S then R
Minimum I/O Cost:
5050
Program end ***
```

Conclusion: The optimizer chooses the join ordering having the lowest estimated I/O cost, thereby reducing disk accesses.

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

The Nested Loop Join compares every tuple of the outer relation with tuples of the inner relation and outputs matching pairs.

Pseudocode:

```
declare r1 = 1
declare r2 = 2
declare r3 = 3

declare s1 = 2
declare s2 = 3
declare s3 = 4

print "Nested Loop Join"
print "Matching tuples are:"

if (r1 == s1)
    print r1
endif

if (r1 == s2)
    print r1
endif

if (r1 == s3)
    print r1
endif

if (r2 == s1)
    print r2
endif

if (r2 == s2)
    print r2
endif

if (r2 == s3)
    print r2
endif

if (r3 == s1)
    print r3
endif

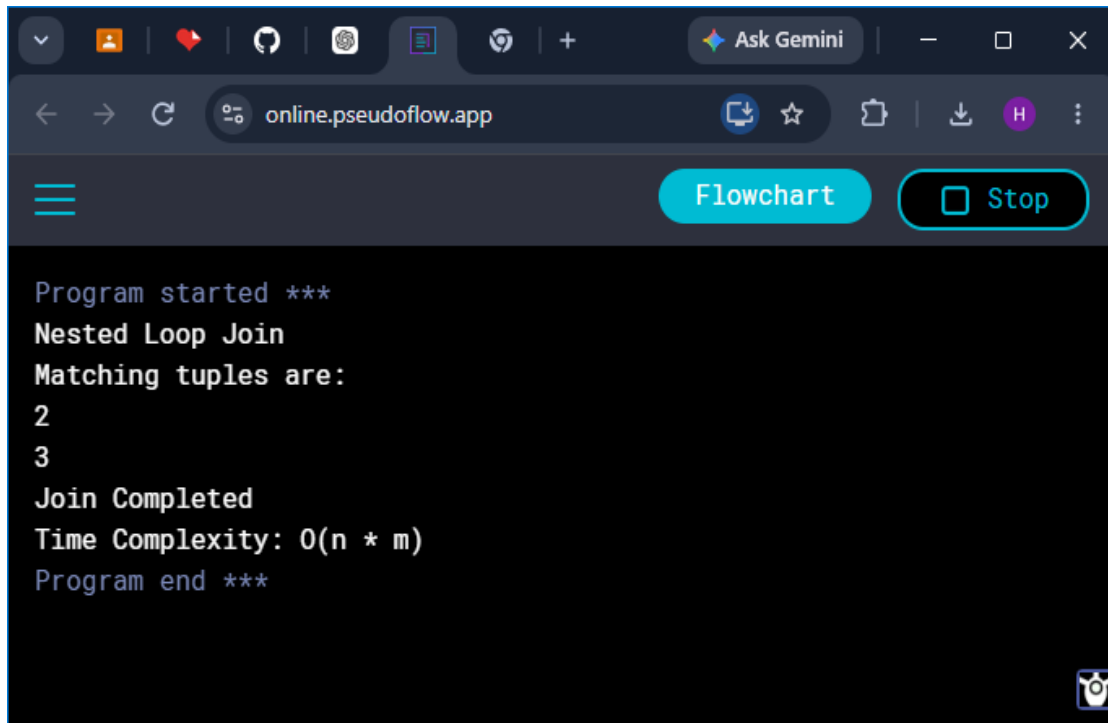
if (r3 == s2)
    print r3
endif

if (r3 == s3)
    print r3
endif

print "Join Completed"
```

```
print "Time Complexity: O(n * m)" END
```

OUTPUT



```
Program started ***
Nested Loop Join
Matching tuples are:
2
3
Join Completed
Time Complexity: O(n * m)
Program end ***
```

Complexity Analysis: If R has T_R tuples and S has T_S tuples, tuple comparisons are $O(T_R \times T_S)$. If R occupies B_R buffer pages and S occupies B_S pages, a basic page-oriented nested loop can require approximately $B_R + B_R \times B_S$ page I/Os (when R is the outer relation).

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

A deadlock exists when the Wait-For Graph contains a cycle. Each node represents a transaction and an edge $T_i \rightarrow T_j$ means T_i is waiting for T_j .

Pseudocode:

```
declare transaction_1_waiting = 1
declare transaction_2_waiting = 1
declare cycle_detected = 0

print "Wait-For Graph Deadlock Detection"

print "Transaction T1 is waiting for T2"
```

```

print "Transaction T2 is waiting for T1"

if (transaction_1_waiting == 1 and transaction_2_waiting == 1)
    declare cycle_detected = 1
endif

if (cycle_detected == 1)
    print "Cycle detected in Wait-For Graph"
    print "DEADLOCK DETECTED"
else
    print "No cycle detected"
    print "NO DEADLOCK"
endif

```

OUTPUT

The screenshot shows a web browser window with the address bar displaying 'online.pseudoflow.app'. The page has a dark theme and a navigation bar with a hamburger menu icon on the left, a 'Flowchart' button in the center, and a 'Stop' button on the right. The main content area displays the following text:

```

Program started ***
Wait-For Graph Deadlock Detection
Transaction T1 is waiting for T2
Transaction T2 is waiting for T1
Cycle detected in Wait-For Graph
DEADLOCK DETECTED
Program end ***

```

Time Complexity: $O(V + E)$, where V is the number of transactions and E is the number of waiting relationships.

Q5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Basic Two-Phase Locking (2PL) has a growing phase in which locks are acquired and a shrinking phase in which locks are released. No new lock is acquired after the transaction starts releasing locks.

Pseudocode:

```
declare lock_granted = 0

declare transaction_state = "GROWING"

print "Basic Two-Phase Locking Protocol"

print "Transaction T1 requests lock on Data Item A"

if (transaction_state == "GROWING")
    declare lock_granted = 1
    print "Lock granted to Transaction T1"
endif

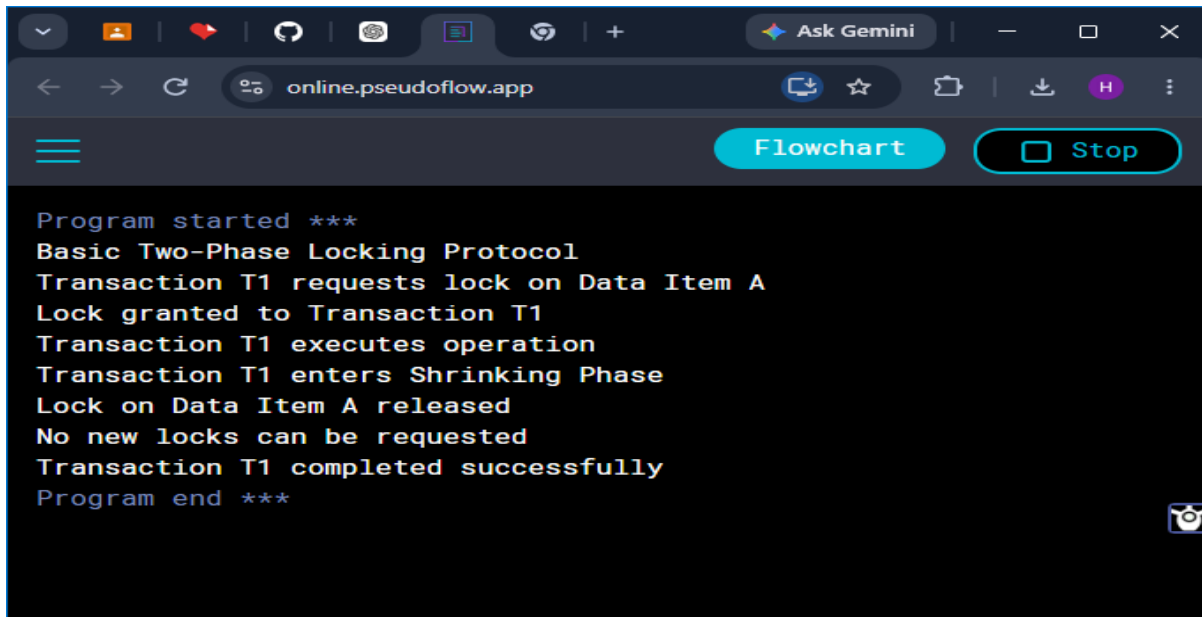
if (lock_granted == 1)
    print "Transaction T1 executes operation"
endif

print "Transaction T1 enters Shrinking Phase"
declare transaction_state = "SHRINKING"

if (transaction_state == "SHRINKING")
    print "Lock on Data Item A released"
    print "No new locks can be requested"
endif

print "Transaction T1 completed successfully"
```


OUTPUT



```
Program started ***
Basic Two-Phase Locking Protocol
Transaction T1 requests lock on Data Item A
Lock granted to Transaction T1
Transaction T1 executes operation
Transaction T1 enters Shrinking Phase
Lock on Data Item A released
No new locks can be requested
Transaction T1 completed successfully
Program end ***
```

Key Rule: Once the transaction enters the shrinking phase, it cannot request any new lock.

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Each transaction receives a unique timestamp. For every data item X, the system maintains Read_TS(X) and Write_TS(X). Thomas' Write Rule allows obsolete writes to be ignored instead of aborting the transaction.

Pseudocode:

```
declare transaction_timestamp = 5

declare read_timestamp = 3

declare write_timestamp = 7

print "Timestamp-Based Concurrency Control"

print "Transaction Timestamp:"

print transaction_timestamp

print "Read Timestamp:"

print read_timestamp

print "Write Timestamp:"
```

```

print write_timestamp

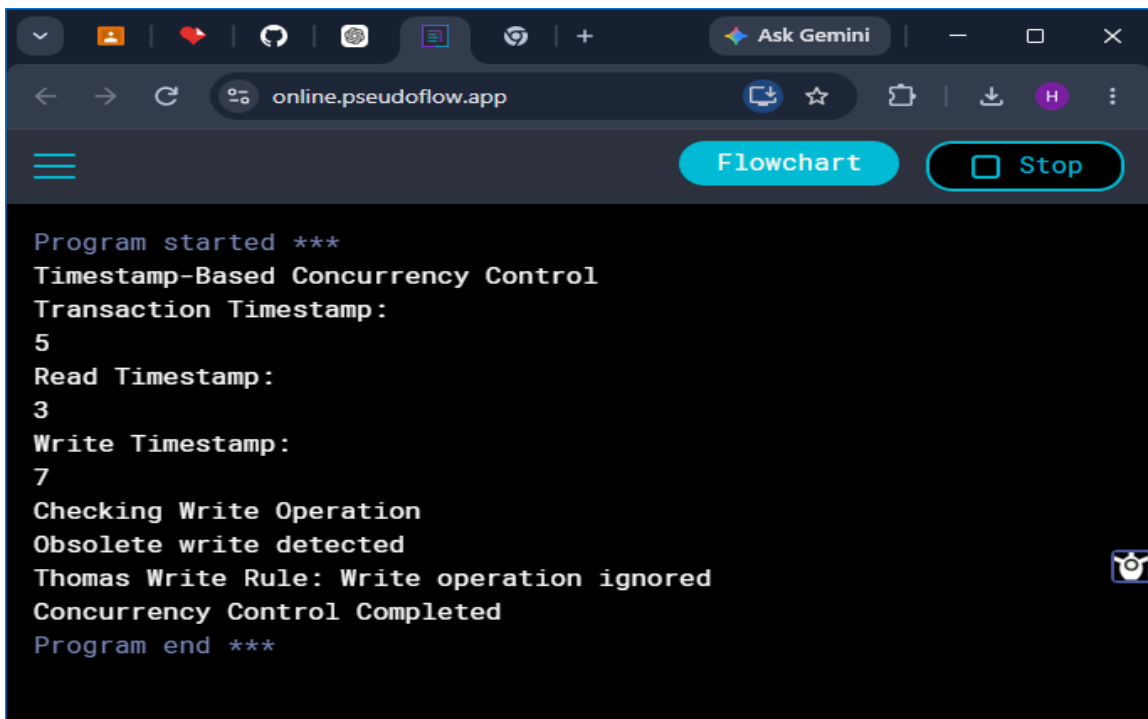
print "Checking Write Operation"

if (transaction_timestamp < read_timestamp)
    print "Transaction aborted because it is older than Read Timestamp"
else
    if (transaction_timestamp < write_timestamp)
        print "Obsolete write detected"
        print "Thomas Write Rule: Write operation ignored"
    else
        print "Write operation executed successfully"
    endif
endif

print "Concurrency Control Completed"
END

```

OUTPUT



```

Program started ***
Timestamp-Based Concurrency Control
Transaction Timestamp:
5
Read Timestamp:
3
Write Timestamp:
7
Checking Write Operation
Obsolete write detected
Thomas Write Rule: Write operation ignored
Concurrency Control Completed
Program end ***

```

Explanation: A write older than an already completed write is obsolete and can be ignored, improving concurrency while preserving timestamp order.

Q7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

ARIES recovery uses write-ahead logging and performs three phases: Analysis to reconstruct transaction state, Redo to repeat history, and Undo to roll back incomplete transactions.

Pseudocode:

```
declare analysis_complete = 0

declare redo_complete = 0

declare undo_complete = 0

print "ARIES Recovery Technique"

print "Phase 1: Analysis"

declare analysis_complete = 1

if (analysis_complete == 1)
    print "Transaction Table Created"
    print "Dirty Page Table Created"
    print "Active transactions identified"
endif

print "Phase 2: Redo"

declare redo_complete = 1

if (redo_complete == 1)
    print "Redoing database updates"
```

```

        print "Database changes restored"
endif

print "Phase 3: Undo"

declare undo_complete = 1

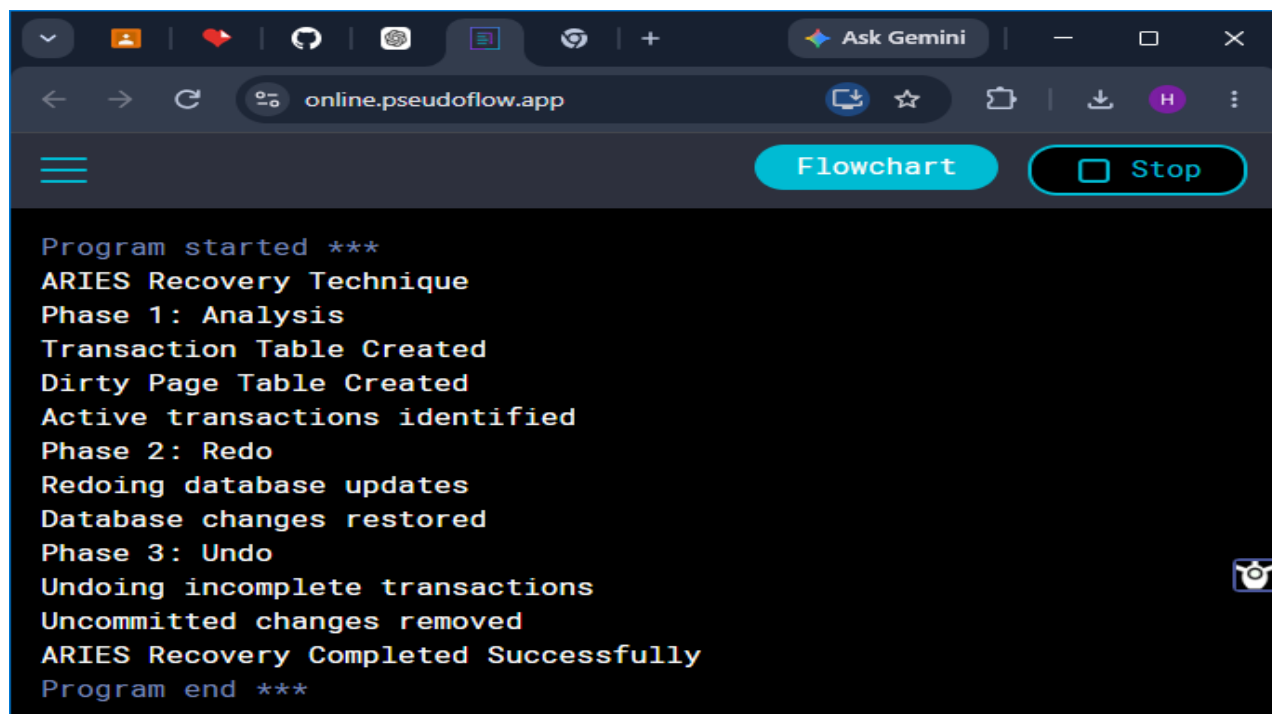
if (undo_complete == 1)
    print "Undoing incomplete transactions"
    print "Uncommitted changes removed"
endif

if (analysis_complete == 1)
    if (redo_complete == 1)
        if (undo_complete == 1)
            print "ARIES Recovery Completed Successfully"
        endif
    endif
endif

endif

```

OUTPUT



```

Program started ***
ARIES Recovery Technique
Phase 1: Analysis
Transaction Table Created
Dirty Page Table Created
Active transactions identified
Phase 2: Redo
Redoing database updates
Database changes restored
Phase 3: Undo
Undoing incomplete transactions
Uncommitted changes removed
ARIES Recovery Completed Successfully
Program end ***

```

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Shadow paging keeps a stable shadow page table and a separate current page table. Updates are made using copy-on-write so the shadow table remains unchanged until commit.

Pseudocode:

```
declare current_page_updated = 0

declare transaction_status = "ACTIVE"

declare system_failure = 1


print "Shadow Paging Recovery"


print "Shadow Page Table contains original data"
print "Current Page Table is created"


print "Updating Current Page"


declare current_page_updated = 1


if (current_page_updated == 1)
    print "New page created successfully"
    print "Current Page Table updated"
endif


print "Checking for System Failure"

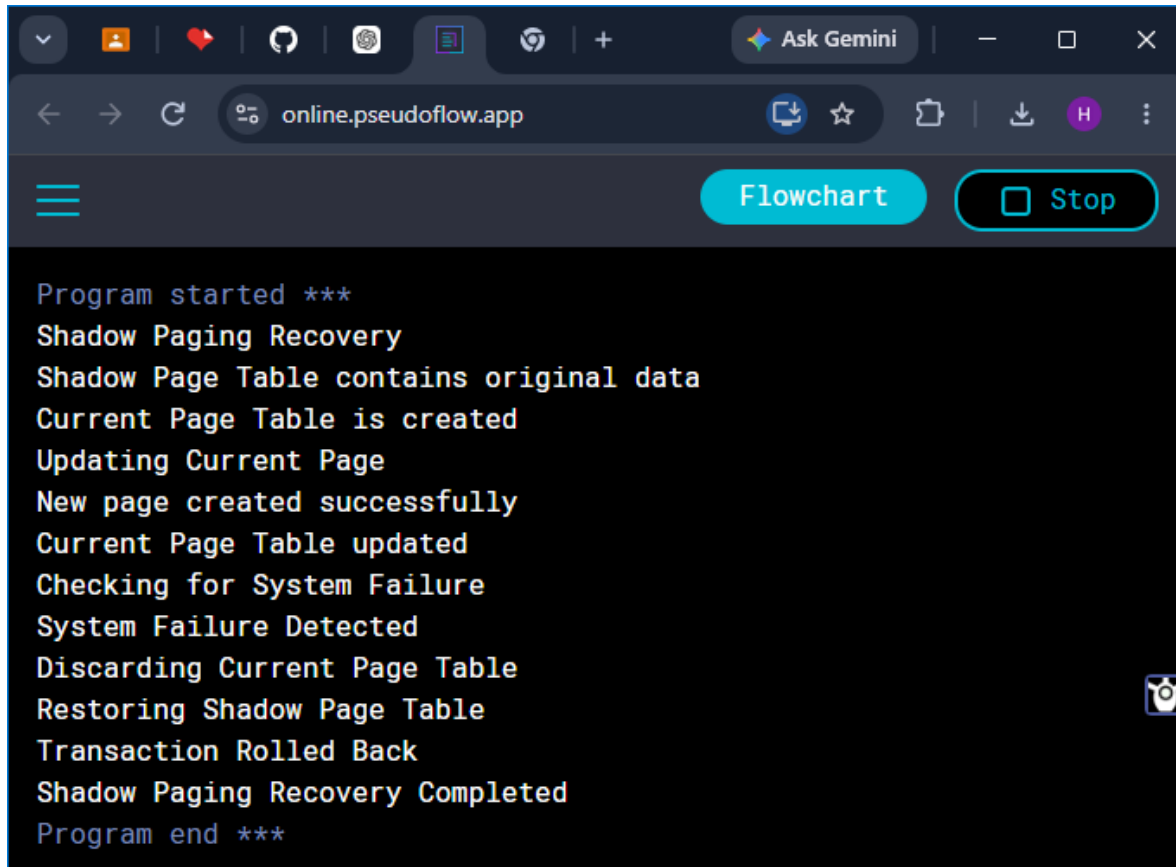

if (system_failure == 1)
    print "System Failure Detected"
    print "Discarding Current Page Table"
    print "Restoring Shadow Page Table"
    print "Transaction Rolled Back"
else
    print "No System Failure"
```

```
    print "Current Page Table becomes permanent"

    print "Transaction Committed"

endif
```

OUTPUT



```
Program started ***
Shadow Paging Recovery
Shadow Page Table contains original data
Current Page Table is created
Updating Current Page
New page created successfully
Current Page Table updated
Checking for System Failure
System Failure Detected
Discarding Current Page Table
Restoring Shadow Page Table
Transaction Rolled Back
Shadow Paging Recovery Completed
Program end ***
```

`print "Shadow Paging Recovery Completed"`**Recovery Idea:** After a failure before commit, the unchanged shadow page table directly points to the old consistent database pages.

Q9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Immediate update permits database pages to be written before commit. Therefore, recovery must REDO committed transactions and UNDO transactions that were incomplete at the time of the crash.

Pseudocode:

```
declare transaction_committed = 1

declare system_crash = 1

declare redo_complete = 0

declare undo_complete = 0


print "Immediate Update Recovery"


print "Transaction updates database"
print "Log records are available"


if (system_crash == 1)
    print "System Crash Detected"


    if (transaction_committed == 1)
        print "Committed transaction found"
        print "Performing REDO operation"
        declare redo_complete = 1
    else
        print "Incomplete transaction found"
        print "Performing UNDO operation"
        declare undo_complete = 1
    endif
endif

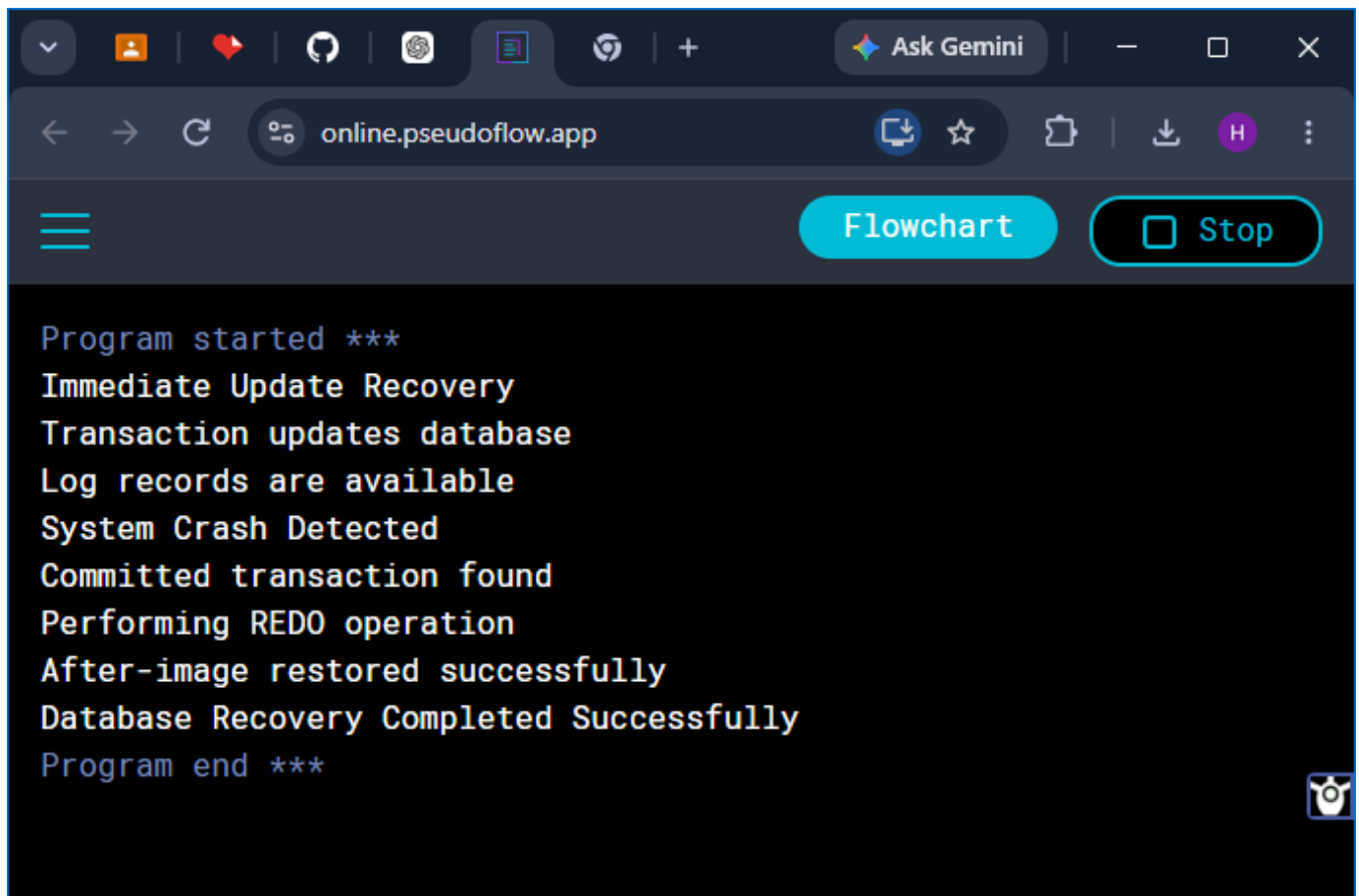

if (redo_complete == 1)
    print "After-image restored successfully"
endif
```

```
if (undo_complete == 1)
    print "Before-image restored successfully"
endif

print "Database Recovery Completed Successfully"RETURN "DATABASE RECOVERED"

END
```

OUTPUT



The screenshot shows a web browser window with the address bar displaying 'online.pseudoflow.app'. The page has a dark theme and a navigation bar at the top with a hamburger menu icon on the left, a 'Flowchart' button in the center, and a 'Stop' button on the right. The main content area displays the output of a program in a monospaced font, with the text as follows:

```
Program started ***
Immediate Update Recovery
Transaction updates database
Log records are available
System Crash Detected
Committed transaction found
Performing REDO operation
After-image restored successfully
Database Recovery Completed Successfully
Program end ***
```

A small robot icon is visible in the bottom right corner of the output area.

Write-Ahead Logging Rule: The log record containing the required before/after information must be safely written before the corresponding database page is written.

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

The connectivity manager reuses connections from a pool, executes the query inside a transaction, commits successful operations, rolls back on exceptions, and always returns the connection to the pool.

Pseudocode:

```
declare connection_available = 1

declare query_success = 1

declare transaction_active = 1


print "Database Connectivity Manager"


print "Checking Connection Pool"


if (connection_available == 1)
    print "Connection obtained from pool"


    print "Executing SQL Query"


    if (query_success == 1)
        print "Query executed successfully"
        print "Committing Transaction"
        declare transaction_active = 0
        print "Transaction Committed"
    else
        print "Query Execution Failed"


        if (transaction_active == 1)
            print "Rolling Back Transaction"
            declare transaction_active = 0
            print "Transaction Rolled Back"
        endif
    endif
endif
```

```
        print "Returning Connection to Pool"

    else

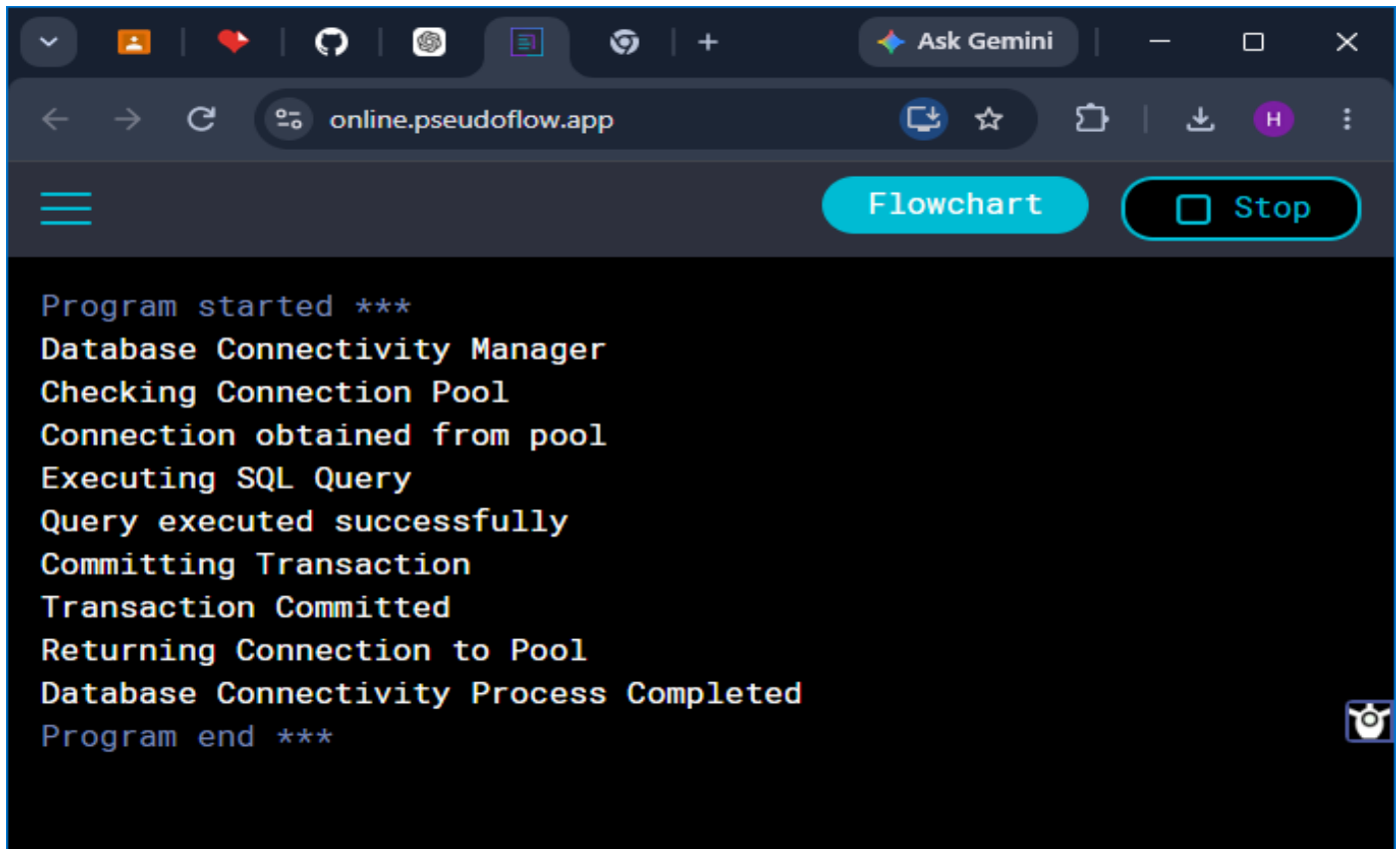
        print "No Connection Available"

        print "Waiting for Connection"

    endif

print "Database Connectivity Process Completed"
```

OUTPUT



The screenshot shows a web browser window with the address bar displaying `online.pseudoflow.app`. The browser's top bar includes a search bar with the text "Ask Gemini" and several navigation icons. Below the address bar, there is a dark blue header with a hamburger menu icon on the left, a cyan button labeled "Flowchart" in the center, and a cyan button labeled "Stop" on the right. The main content area of the browser is a dark blue terminal window displaying the following text:

```
Program started ***
Database Connectivity Manager
Checking Connection Pool
Connection obtained from pool
Executing SQL Query
Query executed successfully
Committing Transaction
Transaction Committed
Returning Connection to Pool
Database Connectivity Process Completed
Program end ***
```

In the bottom right corner of the terminal window, there is a small icon of a person with a speech bubble.

Result: The algorithm supports efficient connection reuse, controlled query execution, exception handling, rollback, and safe resource release.

THANK YOU